

深圳大学实验报告

课程名称： 软件体系结构与设计模式

实验项目名称： 实验 2 OO 设计原则应用

学院： 计算机与软件学院

专业： 软件工程

指导教师： 毛斐巧

报告人： 学号： 班级：

实验时间： 2026 年 4 月 8 日-2026 年 4 月 30 日

实验报告提交时间： 2026 年 4 月 24 日

教务部制

一、实验目的与要求：

- 1.通过场景深入理解和掌握所学的面向对象设计原则。
- 2. 熟练使用面向对象设计原则对系统进行重构。
- 3. 熟练绘制重构后的类图

二、实验内容

AI = 场景 + 数据 + 算法 + 算力，本实验重点在于研究**场景+数据**对于软件体系结构的影响。为实现新增客户需求、变更的客户需求，实验者需要重点思考：

- 1）如何设计、重构第一次作业（或实验）所建的代码仓库。实使得新的代码仓库能够应对新增加的客户需求、变更的客户需求。
- 2）第一作业（或实验）所建的代码仓库是基于什么样的设计原则来构建的？存在什么问题？为什么不能从容应对新需求和需求变更？

新增加的需求如下：

- 1）除 8 个工艺参数以外：缆芯外径、护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度和实际生产速度，还需要收集以下新增参数：**物料品号、创建日期、设备名称、备注信息**。注意这两个字段来自于新的客户提供的 Excel 文件（不同于第一次的作业（或实验））。

批号	数量	芯数	护套芯数	芯公差	材料公差	原始启动/重置/复校	生产订单编号	工单号	物料品名	物料描述	型号	设备名称	工序	生产方式	备注信息	实际加工/创建日期	项目#项目名称	项目记录结果
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	251	班次
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	252	主料(区3)区温度 (实际值) (°C) 130 145 155 161 165 170 175 175
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	253	辅料(区-4)区温度 (实际值) (°C) 180
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	254	挤出速度 (mm)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	255	内护套外径 (mm)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	256	挤出速度 (mm)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	257	生产速度 (米/分)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	258	实际生产速度 (m/min)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	259	编号
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	260	排气总气压 (Mpa)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	261	胶料总气压 (Mpa)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	262	挤出速度 (mm)
01G0100007107701	0.8	24	0	0	19.2	3	FALSE	0240500099-10	AJFK2405301-10	1110100300641	GITA光面(黑色)	GITA-24G.65P-HT-12	伊赛	正常生产	施工 主料手杜伟 *0	2024-6-2 2:52	263	挤出速度 (mm)

- 2）对不符合数据逻辑的生产记录（每个记录的 key 为<批号、物料品号、创建日期>），要存放于独立一张表（无效生产记录表），以便与客户讨论数据时，展示有问题的生产记录。目前了解到的客户关于生产记录的数据逻辑如下：

*缆芯外径 < 护套外径

*挤出内模 < 挤出外模

*缆芯外径、护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度、实际生产速度、**创建日期、设备名称**，这 10 个字段有任何一个为空值，则相应的生产记录无效，必须存入无效生产记录表。

- 3）统计每个物料品号有多多个种不同的工艺参数向量，工艺参数向量定义为< 缆芯外径，护套外径，挤出内模，挤出外模，螺杆速度，螺杆电流，实际生产速度，**设备名称**>。统计每个物料品号的不同工艺参数向量的纯度指标。一个物料品号 P 的一个工艺参数向量 V 的纯度定义为：V 在 P 中出现的次数/P 的所有不同工艺参数向量的数量。

- 4）统计每个物料品号有多少次生产（即不同的批号）

- 5）对于每一个物料品号，统计该物料品号的不同工艺参数向量对应的生产次数，即显示下列格式的信息：

<物料品号、工艺参数向量、生产次数>

6) 统计每个操机手的工艺参数纯度。操机手信息从备注信息中提取, 如“返工 主机手杜伟 跟班黄刚超”, 其中主机手杜伟就是操机手。每个操机手的工艺参数纯度计算方法如下如下:

首先以操作手为 key, 查询出 {<操机手、工艺参数向量、生产次数>}

然后按类似 3) 中计算纯度的方法, 计算每个 key=操作手的纯度。

7) 新增品质检验 Excel 文件, 要求将品质检验数据导入数据库, 以便支持对物料品号的质量进行度量。具体功能需求将在下一个实验给出。

三、理解和分析报告、实践过程及结果等

1. 试分析第一次作业(或实验)代码考虑了哪些面向对象原则? 没有考虑哪些面向对象原则

考虑的原则:

单一职责原则: 代码在宏观上将程序拆分成了几个独立的函数(如 `get_project_root` 负责路径, `isNull` 负责判空, `createRecordExcel` 负责写表头)。

避免重复原则(DRY): 在 `if __name__ == '__main__':` 中, 使用 `for file in files:` 循环来遍历多个文件, 而不是把相同的解析代码复制三遍。

没有考虑的原则:

面向对象思想(未封装): 全篇代码为 0 个类的“面向过程”脚本, 完全没有“对象”的概念。数据结构采用弱类型的字典(dict)传递, 导致数据极其脆弱, 缺乏领域实体(Entity/POJO)的封装。

开闭原则(OCP): 代码充满了硬编码(Hardcode)。例如 `col = 23`(固定读取第 23 列)、`int((sheet.max_row - 1) / 30)`(固定每班次 30 行)、`need_fields` 的硬编码。一旦 Excel 模板列数变动或需求增加, 必须直接修改核心代码, 无法通过扩展(如配置文件或子类)来应对变化。

单一职责原则(SRP): `writeRecordToExcelAndRemoveNull` 方法不仅负责文件 I/O, 还负责业务计算(计算 diff 差值)、数据清洗(调用 `isNull`)、以及数据结构转换, 是典型的“上帝函数/黑盒”, 高耦合。

依赖倒置原则(DIP): 顶层业务逻辑直接依赖于底层的第三方具体库 `openpyxl` 的具体 API 实现。没有定义如 `IDataLoader` 或 `IWriter` 等接口, 导致如果要将 Excel 换成 CSV 或存入数据库, 整个代码必须推翻重写。

2. 为支持 1) ~2) 新增需求, 作业(或实验)1 的哪些代码需求更改? 对改动的代码行进行统计。陈述为什么需要更改。

need_fields 需修改: 增加新增的 4 个字段。

getAllRecords: 原代码死死绑定了“每 30 行一个班次”和“第 23/24 列”的规则。新客户

提供的 Excel 格式不同，原有基于硬索引的解析逻辑将直接崩溃崩溃。

isNull: 判定为空的字段变为特定的 10 个字段，且需加入新的布尔逻辑。

writeRecordToExcelAndRemoveNull / writeRecordToExcel: 原代码只处理了写入一张表的操作。新需求要求对合法记录和无效记录进行分流（无效数据进入独立表，且需要提取联合主键 <批号、物料品号、创建日期>），并在此处加入“外径/内模大小校验”逻辑。

为什么需要更改：

原代码是“面向特定业务场景的流水账脚本”，业务规则（如校验规则、字段列表）与底层实现（如行列遍历、Excel 写入）深度强耦合。由于缺乏“配置隔离”、“规则引擎”和“数据模型”，任何一点需求的增删，都会引发牵一发而动全身的修改。

3. 为支持 3~5) 新增需求，作业（或实验）1 哪些代码可以复用？给出复用的证据？

①基础路径解析工具代码

```
13 def get_project_root():
14     return os.path.dirname(os.path.abspath(__file__))
15
```

这是一个底层 I/O 辅助函数，没有任何业务状态绑定。在需求 3~5 中，无论数据源换成了什么样的新 Excel 文件，依然需要拼接绝对路径来加载文件。该方法符合“单一职责”，可以直接原封不动地被复用到底层 DataLoader 工具类中。

②复杂工艺参数名称（常量字符串）

```
8 # 记录表中需要的字段
9 need_fields = ['批号', '滤芯外径(mm)', '护套外径(mm)', 'diff-输入(d)', '挤出内模(mm)', '挤出外模(mm)', 'diff-挤出(d)',
10               '螺杆速度(rpm)-(挤塑主机速度)(转/分)', '螺杆电流(A)', '实际生产速度(m/min)', '初始行号', '来源文件名']
11
```

需求 3 要求提取由 8 个参数组成的“工艺参数向量”。由于工业 Excel 表头名称非常冗长且包含复杂符号（如 '螺杆速度(rpm)-(挤塑主机速度)(转/分)'），作业 1 中已经将这些易错的硬编码字符串定义好了，可以直接复用这部分字符串字面量（作为字典 Key 或常量配置）。

③基于字典的内存分组聚合逻辑

```
45 if devName not in allRecords.keys():
46     allRecords[devName] = []
47     allRecords[devName].append(oneRecord.copy()) # 将一个班次的信息保存到 allRecords 中
48     oneRecord.clear()
49
50 return allRecords
```

需求 4 要求“统计每个物料品号有多少次生产”；

需求 5 要求统计“该物料品号的不同工艺参数向量对应的生产次数”。

这两个需求本质上都是基于某一特定维度进行 Group By（分组聚合）。作业 1 的原代码中，虽然没有使用类，但利用了 dict 实现了按 devName（设备名称）对数据进行分组归类的基

础算法。

4. 为支持 6) 新增需求，如何复用 3) 的纯度逻辑？

数据清洗层扩展（职责分离）：编写一个独立的解析器工具类 `RemarkParserUtil`，利用正则表达式从备注信息中提取操作手姓名，并将该姓名作为新属性 `operator_name` 存入数据实体对象（`Entity`）中。

泛型统计算法（策略模式/模板方法）：

将纯度计算逻辑从具体的“物料品号”中剥离出来，设计一个泛型的纯度计算器类 `PurityCalculator`。

设计核心方法签名：`calculate_purity(records: List[ProcessRecord], group_by_field: str, vector_fields: List[str])`

当满足需求 3 时：调用 `calculate_purity(data, group_by_field="material_code", vector_fields=[...])`

当满足需求 6 时：调用 `calculate_purity(data, group_by_field="operator_name", vector_fields=[...])`

5. 如何设计/重构代码仓库，以支持需求 7)？

定义实体层（`Entity / POJO`）：新增 `ProcessRecordEntity`（对应工序）和 `QualityInspectionEntity`（对应品检），封装各自的属性。

抽象数据加载层（`Data Loader 接口`）：定义接口 `IDataLoader`，包含方法 `load_data(file_path)`。

编写实现类 `ProcessRecordExcelLoader` 和 `QualityInspectionExcelLoader`。业务主控只依赖 `IDataLoader` 接口，不知道具体的解析细节，实现依赖倒置原则。

抽象规则校验引擎（`Validation Engine`）：定义验证规则接口 `IRule`。

对于品检数据，实现类似 `QualityThresholdRule` 等具体的规则类，注册到验证引擎中。

持久化层（`DAO / Repository`）：构建数据库（`MySQL`），建立对应的表结构（`tb_process_record`, `tb_quality_record`，可通过批号或物料品号关联）。

使用 ORM 框架（如 `SQLAlchemy/MyBatis`）或编写通用 DAO 层进行数据持久化，彻底抛弃原来“读 Excel -> 算 -> 写 Excel”的落后模式。

6. 设计应支持变化----- 试问你的代码哪些设计能够支持 1)~7) 的变化？

实体建模（支持需求 1、7）：将弱类型的字典替换为强类型的 `Entity` 类。当需要增加字段（需求 1）或引入全新的数据域（需求 7）时，只需增加新的实体类或类属性即可，不影响原有逻辑。

策略模式构建规则引擎（支持需求 2）：将“非空校验”、“内模外模大小对比校验”等逻辑封装为独立的实现了 `IRule` 接口的校验类（如 `NotNullRule`、`SizeCompareRule`）。当新增无

效记录判定条件时（需求 2），只需新增一个 Rule 类并注入引擎，无需修改原有的主流程代码，完美契合开闭原则 (OCP)。

MVC/分层架构与 DAO 层引入（支持需求 3、4、5）：将数据统一落盘至关系型数据库。通过独立的 StatisticsService 层和 DAO 层的 SQL 聚合能力（GROUP BY material_code），极大地简化了统计频次和分组操作，使复杂的指标计算变为简单的数据库查询。

行为参数化与泛型设计（支持需求 6）：将统计纯度提取为与具体业务字段无关的通用工具方法（PurityCalculator），通过传入不同的动态字段名（如将“品号”换为“操作手”），轻松支持任意维度的纯度分析，体现了单一职责 (SRP)和代码复用 (DRY)。

四、实验总结与体会

(写写感想、建议等)

本次《软件体系结构与设计模式》的重构实验，让我经历了一次从“单纯追求代码能跑”到“追求代码好维护、易扩展”的思维蜕变。

1. 深刻认识到了“面向字典编程”与“面条式代码”的危害

在分析第一次作业的代码时，我发现虽然代码逻辑能够跑通，但全部揉捏在几个巨大的函数中，且数据流转完全依赖弱类型的字典(dict)。这在面临新需求（如新增字段、按新维度统计）时，暴露出极大的脆弱性——缺乏类型提示、极易引发 KeyError、业务逻辑与底层 I/O 深度耦合。这让我真切体会到：在工业级项目中，没有面向对象封装的代码，其技术债务是呈指数级增长的。

2. 设计模式与 SOLID 原则是应对需求变更的“利器”

在面对本次实验多达 7 项的新增和变更需求时，如果直接在旧代码上打补丁，必然会导致系统崩溃。通过重构，我切实感受到了体系结构设计魅力：

开闭原则 (OCP) 与策略模式的结合：通过抽象出 IValidationRule 接口，把复杂的“内模外模大小对比”、“字段非空判断”拆分为独立的类。当面临需求 2（引入新的无效判定规则）时，我只需新增一个实现了该接口的类即可，主流程代码做到了“零修改”。

单一职责 (SRP) 与抽象复用：在处理需求 3（物料品号纯度）和需求 6（操作手纯度）时，我没有复制粘贴两套代码，而是将算法抽象为泛型服务（传入不同的 group_by 字段）。这让我明白了，好的设计不仅能解耦，还能极大提升代码的复用率。

3. 数据处理脚本同样需要严谨的软件架构

以往我有一种误区，认为只有开发大型 Web 系统或 App 才需要用到高大上的设计模式，写 Python 数据清洗脚本只需“面向过程”堆代码即可。但本次工业数据预处理的场景给我上了一课：AI 时代的数据处理系统，面临着极其频繁的数据源变更和指标维度变更。必须建立稳固的实体层 (Entity)、数据访问层 (Loader/DAO) 和业务逻辑层 (Service)，才能从容应对“明天突然换一个 Excel 模板”或“新增品检数据导入”这样的突发需求。

五、成绩评定及评语

1.指导老师批阅意见:

2.成绩评定:

指导教师签字:毛斐巧

2026 年 月 日